

BudgetFlow - Project Documentation

1. Project Overview

BudgetFlow is a comprehensive full-stack web application designed for personal financial management. It empowers users to track their expenses, manage budgets, monitor net worth, and gain insights into their financial health through interactive charts and simulators. The platform provides a secure environment for per-user data isolation.

2. Key Features

🔒 Security & Authentication

- **Secure User Registration & Login:** Custom authentication system with hashed passwords.
- **Email OTP Verification:** Ethereum-powered email verification flow during user onboarding to validate and activate accounts.
- **JWT-based Sessions:** Access management using JSON Web Tokens.
- **Data Isolation:** Users only have access to their own private financial records.

💰 Financial Management

- **Transactions Management:** Add, edit, delete, and view income and expense transactions.
- **Accounts Tracking:** Manage multiple accounts (bank accounts, cash, etc.).
- **Debts & Credit Cards:** Track outstanding debts and credit card balances.
- **Budgets:** Set and monitor budget limits for different categories.
- **Net Worth Tracking:** Calculate and visualize the user's overall net worth over time.

📊 Analytics & Insights

- **Dashboard Overview:** A high-level summary of the user's current financial status.
- **Interactive Charts:** Visualizations for income vs. expenses, categorical breakdowns, and historical data.
- **Financial Simulator:** Simulate different financial scenarios and predict future financial health.
- **Calendar View:** View transactions mapped on a calendar format to track daily expenses easily.

⚙️ Advanced Tools

- **Scheduled Transactions:** Plan and manage recurring expenses or incomes.
- **Bank Synchronization (Mock/Integrations):** Interface to sync transactions with external bank accounts.
- **Data Import/Export:** Users can import financial data or export their records for offline use.

3. Technology Stack

Frontend (Client-side)

The frontend is built for performance and a modern user experience.

- **Framework:** React 18 with TypeScript.
- **Build Tool:** Vite (for fast module replacement and builds).
- **Styling:** Tailwind CSS combined with `shadcn/ui` (Radix UI primitives) for accessible and highly customizable components.
- **Routing:** React Router DOM (v6).
- **Form Handling & Validation:** React Hook Form coupled with Zod.
- **Data Fetching & Caching:** TanStack React Query.
- **Data Visualization:** Recharts (for dynamic charts and graphs).
- **Icons:** Lucide React.
- **Date Management:** `date-fns` & `react-day-picker`.

Backend (Server-side)

A robust and secure backend API.

- **Runtime:** Node.js.
- **Framework:** Express.js.
- **Database:** MongoDB (Object Data Modeling via Mongoose).
- **Authentication:** `jsonwebtoken` (JWT) for secure authentication.
- **Security:** `bcryptjs` for secure password hashing.
- **Middleware:** CORS and generic error handling.

4. Architecture

- **Client-Server Model:** The application follows a separation of concerns principle with a decoupled React frontend and an Express RESTful API backend.
- **RESTful API:** Standardized endpoints for authentication (`/api/auth`) and resource manipulation (`/api/transactions`, etc.).
- **State Management:** Uses React Query for server-state synchronization globally, and standard React hooks for component-level UI states.

5. Future Roadmap

- Fully automated bank syncing endpoints using API providers (e.g., Plaid).
- Deep AI insights for personal saving recommendations based on spending trends.
- Comprehensive recurring automated billing tracking.